# Generative AI

Prompt Engineering for Software Developers

**Day 1 - Foundations of LLMs, NLP & OpenAI**

KavinSchool

# Welcome!

- Introductions: your name, role, and one thing you hope to learn
- Course objectives and 4-day roadmap
- Lab environment check: Python, VS Code, API keys

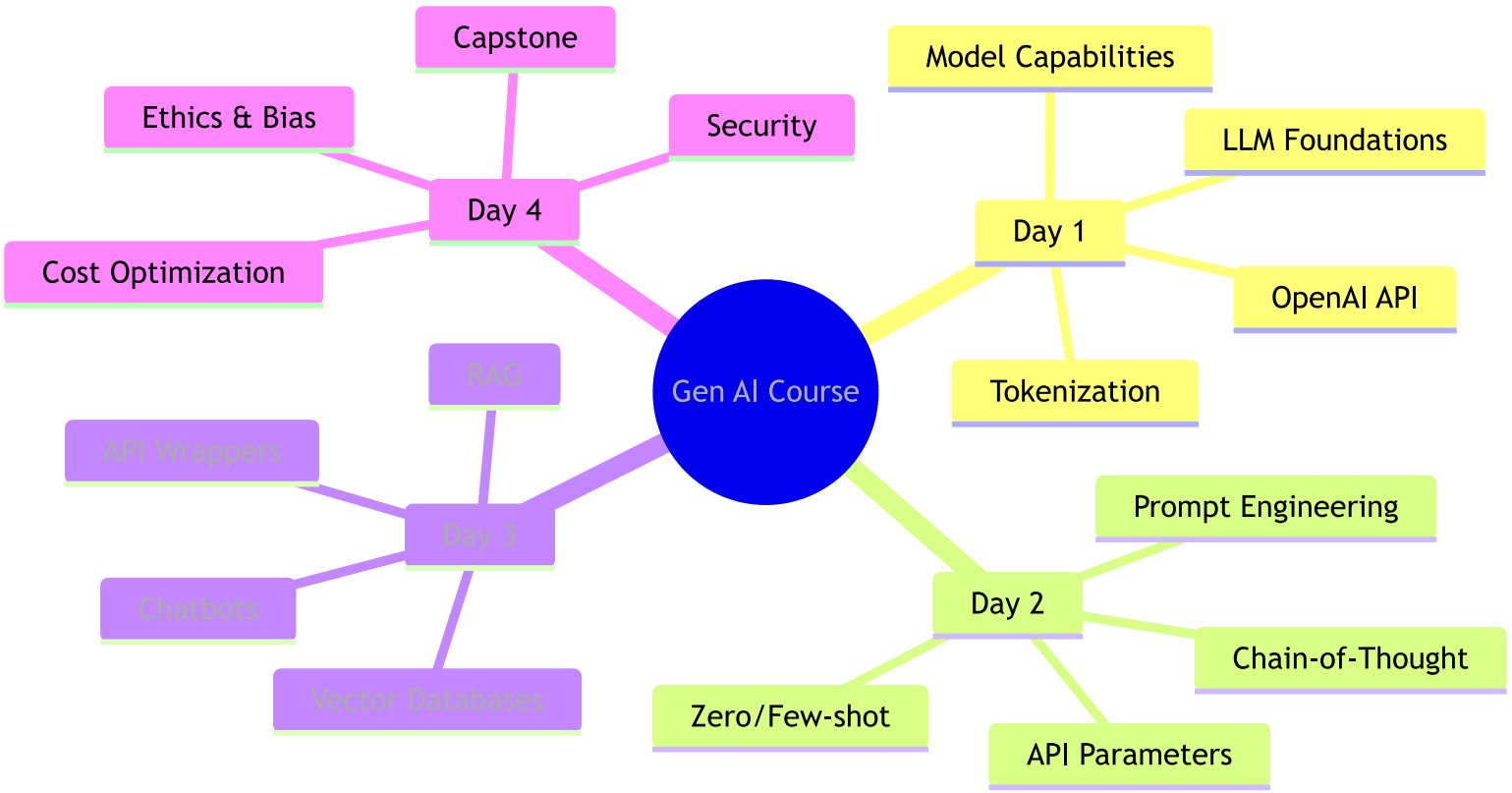
> **By end of Day 1:** You will be able to call an LLM API, understand how tokens work, and reason about model capabilities.

# Daily Schedule

| Block | Duration | Notes |
|---|---|---|
| Session 1 (Morning, Part 1) | ~1 hr 45 min | |
| **Morning Break** | **15 min** | |
| Session 2 (Morning, Part 2) | ~1 hr 45 min | |
| **Lunch Break** | **45 min** | |
| Session 3 (Afternoon, Part 1) | ~1 hr 45 min | |
| **Evening Break** | **15 min** | |
| Session 4 (Afternoon, Part 2) | ~1 hr 30 min | Wrap-up + Q&A |

Total instructional time per day: **~6 hr 45 min**

# Course Overview: 4 Days

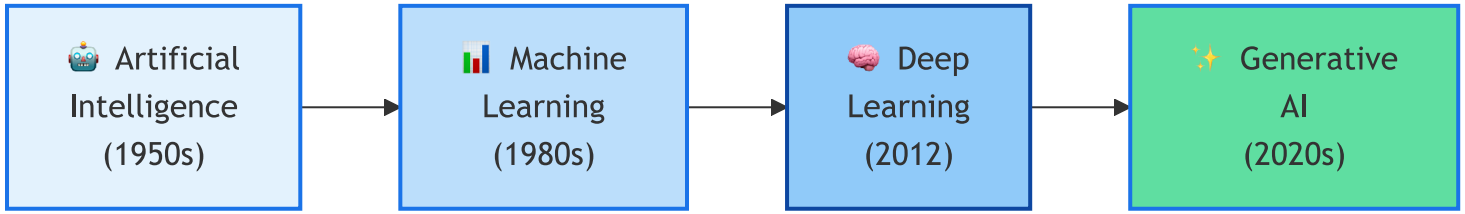


# The Evolution of AI

## From Rule-Based Systems to Generative Models

KavinSchool

# AI → ML → DL → Generative AI




| Era | Key Idea | Example |
|---|---|---|
| **AI** (1950s) | Rule-based, expert systems | Chess engines, spell checkers |
| **ML** (1980s) | Learning from data | Spam filters, recommendations |
| **DL** (2012) | Neural networks at scale | Image recognition, voice assistants |
| **Gen AI** (2020s) | Creating new content | ChatGPT, DALL-E, GitHub Copilot |

# What Changed? Why Now?

Three breakthroughs converged:

## 1. The Transformer Architecture (2017)

- "Attention Is All You Need" paper by Google
- Enabled parallel processing of entire sequences
- Replaced sequential RNNs/LSTMs

## 2. Scale: Compute + Data

- Models grew from millions to trillions of parameters
- Trained on the entire internet

# What Changed? Why Now?

## 3. RLHF: Human Alignment

- Reinforcement Learning from Human Feedback
- Made models helpful, harmless, and honest

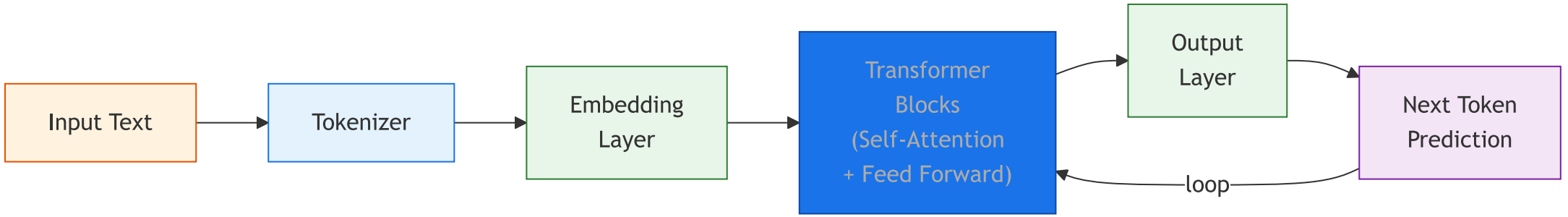
# What Are LLMs?

## Large Language Models: The Engine Behind Gen AI

KavinSchool

# Large Language Models: Big Picture

An LLM is a neural network that **predicts the next token** given a sequence of previous tokens.

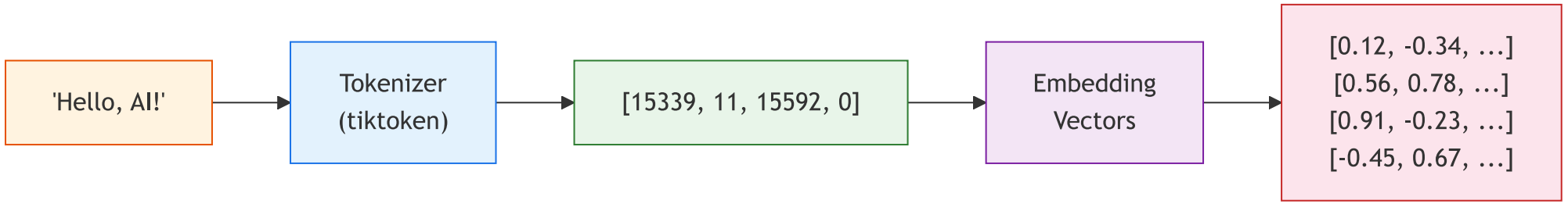



> LLMs don't "understand" language. They are extremely good at statistical pattern matching over text.

# Core Concepts

| Concept | What It Means | Analogy |
|---|---|---|
| **Token** | A chunk of text (~4 chars in English) | A syllable |
| **Embedding** | A vector (list of numbers) representing a token | GPS coordinates for a word |
| **Context Window** | Max tokens the model can see at once | Working memory |
| **Attention** | How the model weighs relationships between tokens | Highlighting key words |
| **Parameters** | The "knowledge" stored in the model weights | Synapses in a brain |

# Tokenization: How LLMs Read Text




## Why it matters:

- You're **billed per token**, not per word
- Context windows are measured in **tokens**
- Different languages tokenize differently (Chinese = more tokens per character)

# Tokenization: Code Example

```
import tiktoken
enc = tiktoken.encoding_for_model("gpt-4o")
tokens = enc.encode("Hello, AI!")
print(f"Tokens: {tokens}, Count: {len(tokens)}")
# Tokens: [15339, 11, 15592, 0], Count: 4
```

# Working Memory (Context Window)

| Model | Context Window | Pages of Text (~500 words/page) |
|---|---|---|
| GPT-4o | 128K tokens | ~200 pages |
| GPT-4o-mini | 128K tokens | ~200 pages |
| Claude 3.5 Sonnet | 200K tokens | ~310 pages |
| Gemini 2.5 Pro | 1M tokens | ~1,500 pages |
| Llama 3.1 | 128K tokens | ~200 pages |

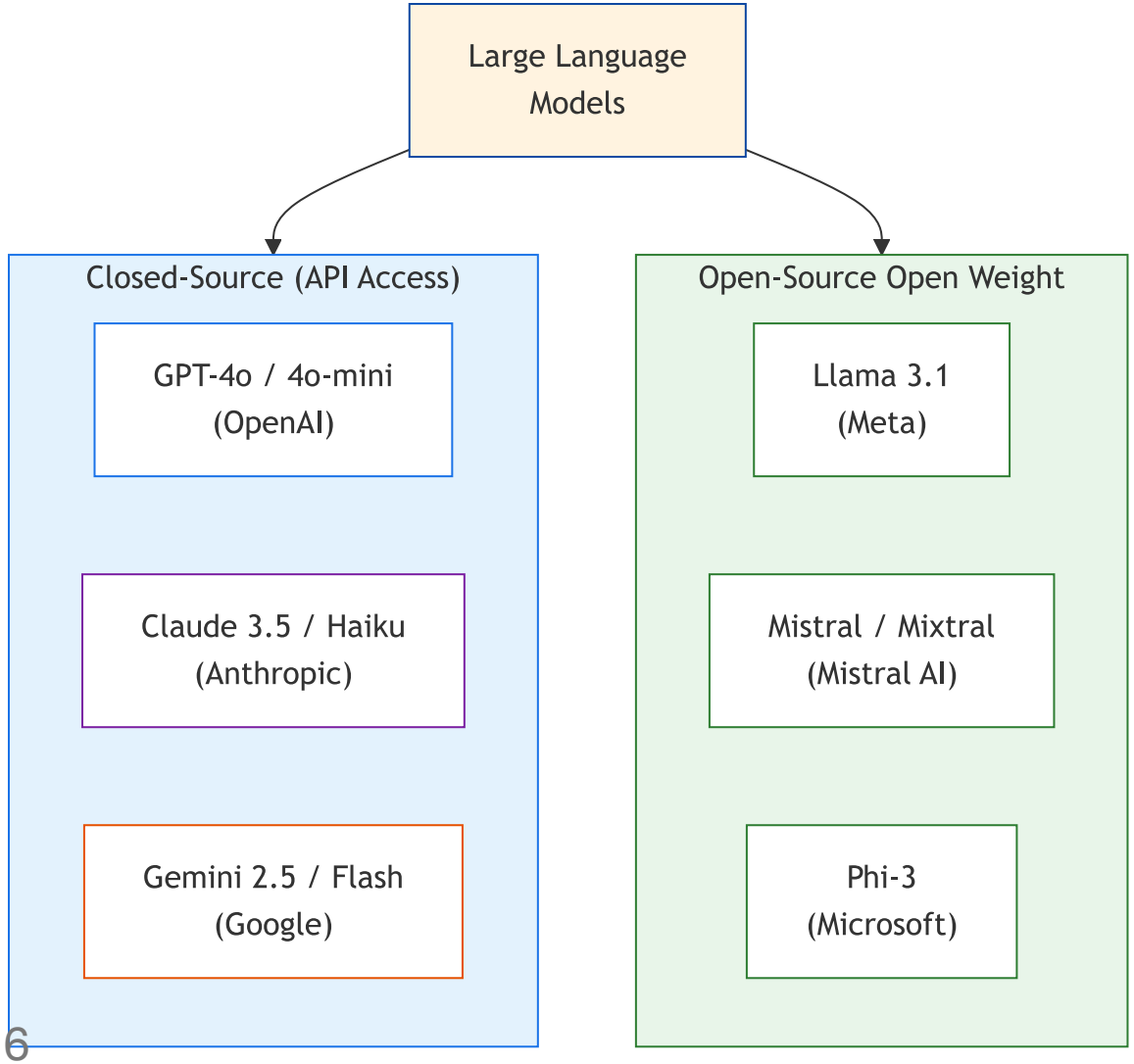
> Larger context window ≠ better results. Models still struggle with "needle in a haystack", finding specific information in long contexts.

# The LLM Landscape

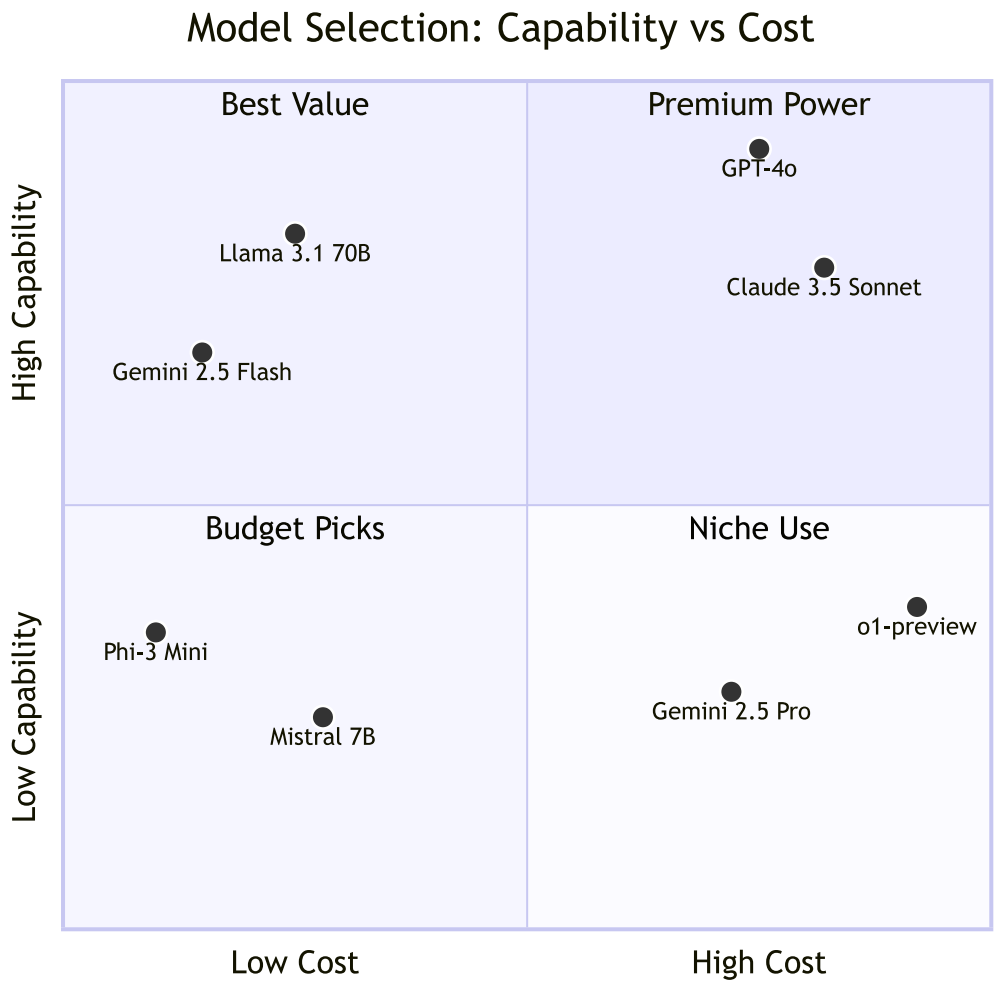
## Choosing the Right Model

KavinSchool

# Open vs Closed Models



# Model Comparison: Capability vs Cost



# When to Use Which?

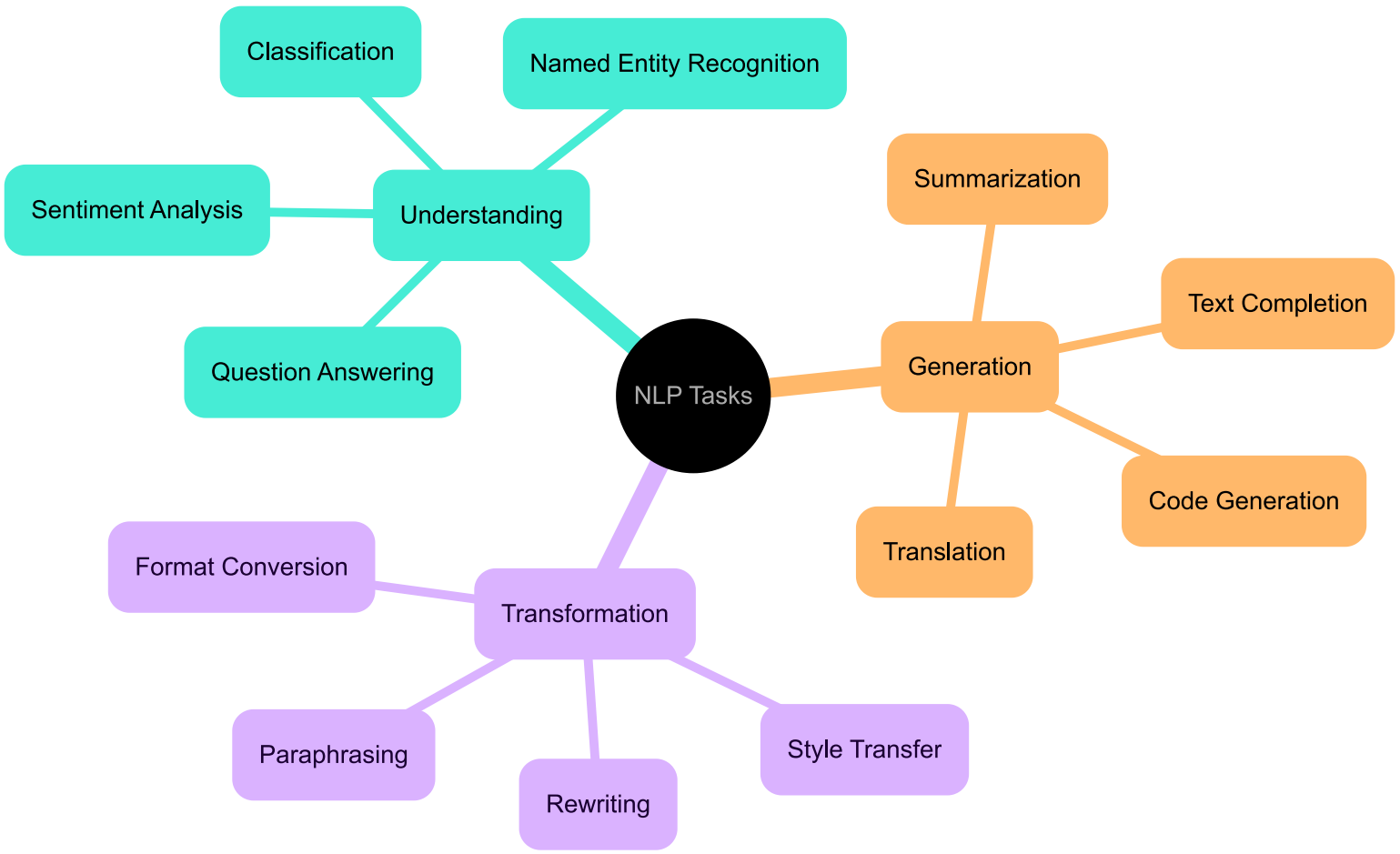
| Use Case | Recommended | Why |
|---|---|---|
| Quick prototyping | GPT-4o-mini, Gemini Flash | Fast, cheap |
| Production quality | GPT-4o, Claude Sonnet | Best reasoning |
| Long documents | Gemini 2.5 Pro | 1M token context |
| On-premise / privacy | Llama 3.1, Mistral | Self-hosted, no data leaves |
| Code generation | GPT-4o, Claude Sonnet | Top benchmarks |
| Budget-sensitive | Open-source + local | Zero API cost |

# NLP Refresher

## What Can LLMs Actually Do?

KavinSchool

# NLP Task Categories



# Understanding Tasks: Examples

**Classification:** Is this email spam or not?

```
Input:  "Congratulations! You've won a free iPhone!"
Output: "spam" (confidence: 0.97)
```

**Named Entity Recognition (NER):** Find the entities

```
Input:  "Apple CEO Tim Cook announced the iPhone 16 in Cupertino."
Output: ORG=Apple, PERSON=Tim Cook, PRODUCT=iPhone 16, LOC=Cupertino
```

**Sentiment Analysis:** What's the tone?

```
Input:  "The new update completely broke my workflow."
Output: NEGATIVE (confidence: 0.91)
```

# Generation Tasks: Examples

## Summarization

```
Input:  [500-word article about quantum computing]
Output: "Quantum computing uses qubits to perform parallel
         calculations, potentially solving problems that are
         intractable for classical computers."
```

## Code Generation

```
Input:  "Write a Python function to find the nth Fibonacci number"
Output: def fibonacci(n): ...
```

## Translation

```
Input:  "The meeting is at 3 PM" → French
Output: "La réunion est à 15 heures"
```

# Demo Time

## Tokenizer Visualization with `tiktoken`

KavinSchool

# Demo: Tokenizer Visualization (Part 1)

```
import tiktoken

# Compare tokenizers across models
for model in ["gpt-4o", "gpt-4o-mini", "gpt-4"]:
    enc = tiktoken.encoding_for_model(model)
    text = "Generative AI is transforming software development."
    tokens = enc.encode(text)
    print(f"{model:15} → {len(tokens)} tokens: {tokens}")
```

# Demo: Tokenizer Visualization (Part 2)

```
# See how different languages tokenize differently
texts = [
    "Hello, world!",              # English
    "Bonjour le monde!",          # French
    "こんにちは     ",             # Japanese
    "def hello(): return 'world'" # Code
]
enc = tiktoken.encoding_for_model("gpt-4o")
for text in texts:
    tokens = enc.encode(text)
    print(f"{len(tokens):3} tokens: {text}")
```

> Try it yourself! Open `day1/demos/demo-tokenizer.py`

# Session 1: Key Takeaways

1. **Gen AI builds on decades of AI/ML progress.** Transformers + scale + RLHF = breakthrough
2. **LLMs predict the next token.** They don't truly "understand" but they're remarkably capable
3. **Tokens are the currency.** You pay per token, context windows are in tokens
4. **Attention is the secret sauce.** It lets models understand word relationships
5. **Choose the right model.** Balance capability, cost, context window, and privacy needs

> Next: Session 2, hands-on with the LLM API. You'll make your first API call!

# Questions?

Session 1 !

KavinSchool